Generador de funciones DDS con STM32H503CBT6

Diseño, decisiones de ingeniería y diagramas de flujo

1. Resumen del diseño

Parámetro	Valor
Microcontrolador	STM32H503CBT6 (Cortex-M33, 250 MHz, 128 KB flash, 32 KB RAM)
Reloj de muestreo	TIM2 → TRGO, $F_s = 250\,\text{MHz}/250 = 1\,\text{MS/s}$
Salidas	DAC1 CH1 (PA4) y CH2 (PA5), modo dual, 12 bits
Transferencia	GPDMA1 canal 0, circular por lista enlazada, buffer ping-pong 2×256
Acumulador de fase	32 bits ⇒ resolución $F_s/2^{32} = 0{,}233\,\text{mHz}$
Tabla	4096 muestras × 12 bits (en `uint16_t`), doble buffer en RAM
Rango de salida	0,1 Hz a 400 kHz ($0{,}4\,F_s$)
Interpolación	lineal, automática para $f < F_s/N = 244\,\text{Hz}$
Encoders	TIM1 (frecuencia) y TIM3 (forma de onda), modo encoder por hardware
Formas de onda	seno, triangular, sierra ±, cuadrada, seno rectificado, arbitraria

2. Arquitectura

La idea central es que **el CPU nunca escribe en el DAC**. El timer marca el ritmo con una precisión de cristal, el DAC convierte en cada flanco de TRGO y el GPDMA le entrega la muestra siguiente. El CPU solo recalcula la mitad del buffer que el DMA ya terminó de leer. Así el jitter de muestreo es nulo, sin importar las interrupciones que haya.

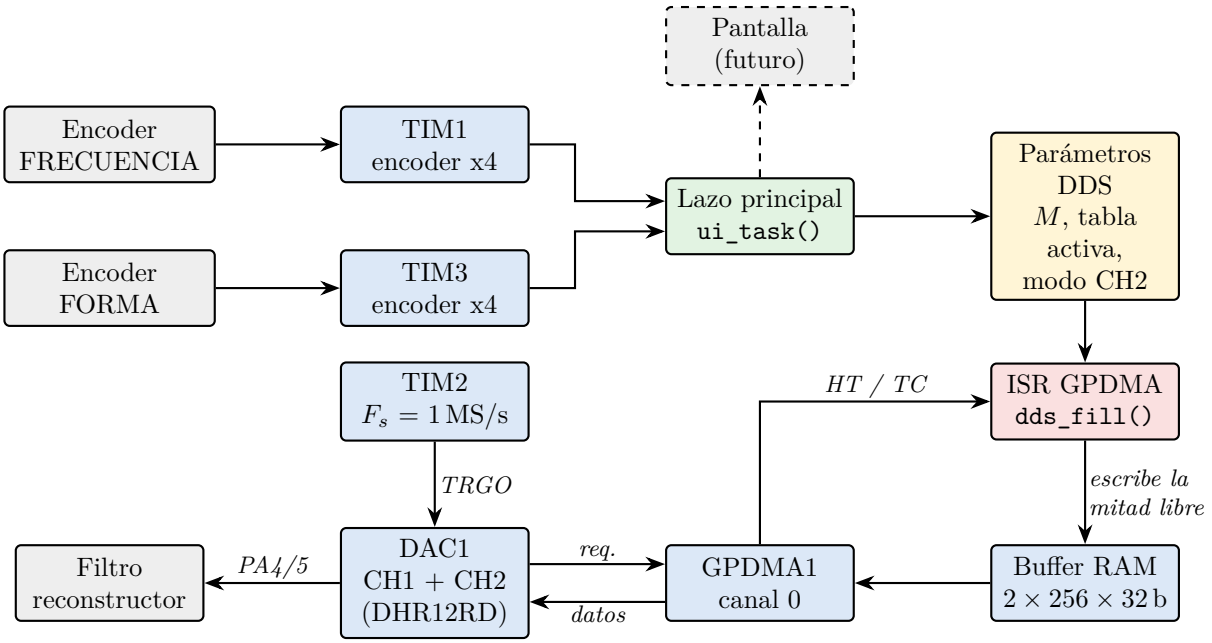


Figura 1: Diagrama en bloques del generador.

3. Decisiones de diseño

3.1. ¿TIM1 o TIM2?

En el STM32H503 el DAC puede dispararse desde TIM1, TIM2, TIM3, TIM6, TIM7 y LPTIM1/2 (campo `TSELx` de `DAC_CR`). Todos los timers corren a 250 MHz con el prescaler de APB en 1, así que la resolución de F_s es la misma con cualquiera. La diferencia está en lo que se pierde al usarlo:

Timer	Características	Uso en este diseño
TIM1	Avanzado, 16 bits, PWM complementario con tiempo muerto, el único del chip	**Encoder de frecuencia** (PA8/PA9). Usarlo solo como reloj desperdiciaría sus funciones.
TIM2	32 bits, 4 canales, dispara al DAC (`TSEL`=2)	**Reloj de muestreo del DDS** (elegido)
TIM3	16 bits, 4 canales, modo encoder	**Encoder de forma de onda** (PA6/PA7)
TIM6	Básico, sin pines, pensado para el DAC (`TSEL`=5)	Alternativa equivalente: `DDS_TIMER_TIM6` en `dds_config.h` libera TIM2

Para generar el TRGO, TIM1 y TIM2 dan el mismo resultado. Elegí **TIM2** porque TIM1 es el único timer avanzado del chip y su par de pines PA8/PA9 sirve muy bien para el encoder. Hacen falta dos timers con modo encoder (TIM1 y TIM3), y con TIM2 en el DDS no se desperdicia ninguna función especial.

3.2. Frecuencia de muestreo y frecuencia máxima

$$F_s = \frac{f_{TIM}}{ARR+1} = \frac{250\,\text{MHz}}{250} = 1\,\text{MS/s} \qquad M = \text{round}\left(\frac{f_{out}\cdot 2^{32}}{F_s}\right) \qquad \Delta f = \frac{F_s}{2^{32}} = 0{,}233\,\text{mHz} \quad (1)$$

Elegí 1 MS/s porque es la velocidad de actualización típica que especifica ST para sus DAC con el buffer de salida activado (confirmalo en la tabla *DAC characteristics* del DS14053). El límite teórico es Nyquist, $F_s/2 = 500\,\text{kHz}$. En la práctica conviene $f_{max} = 0{,}4\,F_s = 400\,\text{kHz}$, porque la primera imagen cae en $F_s - f$ y el filtro reconstructor necesita banda de transición. La retención de orden cero atenúa según $\text{sinc}(f/F_s)$: a 400 kHz se pierden 2,4 dB. Para un seno limpio sin un filtro muy abrupto, conviene quedarse por debajo de $F_s/4 = 250\,\text{kHz}$. Las formas con armónicos (cuadrada, sierra) se ven bien hasta $\approx F_s/20$.

Si el DAC de tu placa lo soporta, `DDS_FS_HZ` acepta 2; 2,5 y 5 MS/s, que son divisores exactos de 250 MHz. Por encima de 1 MS/s usá `MODE = 010` (buffer interno OFF) y un amplificador operacional rápido como buffer externo. El CPU acompaña: a 5 MS/s el lazo directo usa ≈ 25 % del tiempo.

3.3. Tamaño de tabla recomendado: 4096×12 bits

En un DDS, N **no** es la cantidad de muestras por período que salen. Esas son siempre F_s/f_{out}. N fija la *precisión de fase*. Al truncar la fase a $\log_2 N$ bits aparecen espurias de aproximadamente $\text{SFDR} \approx 6{,}02 \cdot \log_2 N$ dBc.

N	RAM (2 buffers)	SFDR trunc.	Error seno interp.	Sierra	Comentario
256	1 KB	$\approx$ 48 dBc	0,15 LSB	16 LSB/paso	solo con interpolación
1024	4 KB	$\approx$ 60 dBc	0,01 LSB	4 LSB/paso	aceptable
4096	**16 KB**	**$\approx$ 72 dBc**	**0,0006 LSB**	**1 LSB/paso**	**recomendado**
16384	64 KB	$\approx$ 84 dBc	–	1 LSB/paso	no entra en 32 KB

Recomendación: 4096 muestras de 12 bits guardadas en `uint16_t` (8 KB por tabla), por estas razones:

- 72 dBc de SFDR coinciden con el techo de un DAC ideal de 12 bits ($6{,}02 \cdot 12 + 1{,}76 = 74$ dB). Una tabla más grande no mejora la salida porque manda el DAC.
- Una sierra de 4096 pasos recorre los 4096 códigos: 1 LSB por muestra, sin pérdida.
- El índice se obtiene con un desplazamiento (`fase >> 20`), sin módulo.
- 12 bits justificados a la derecha van directo al registro `DHR12RD`.
- Con doble buffer (16 KB) el cambio de forma de onda no produce glitches, y todavía sobra la mitad de la RAM.
- Se agrega una muestra guarda ($T[4096] = T[0]$) para que la interpolación lea $T[i+1]$ sin máscara.

Las tablas se calculan en el arranque (seno, triangular, etc.). Las formas arbitrarias se "graban" en flash con `tools/gen_tabla.py`, a partir de una fórmula o de un CSV medido, y se copian a RAM al seleccionarlas.

3.4. Interpolación lineal en baja frecuencia

Cuando $f_{out} < F_s/N$ (244 Hz), el paso de fase es menor que una muestra de tabla y cada valor se repite varias veces. El resultado es una escalera. En ese caso se usan los 12 bits de fracción que siguen al índice:

$$\underbrace{\texttt{fase}[31{:}20]}_{i} \quad \underbrace{\texttt{fase}[19{:}8]}_{\phi} \qquad y = T[i] + \left\lfloor \frac{(T[i+1] - T[i]) \cdot \phi + 2^{11}}{2^{12}} \right\rfloor \tag{2}$$

El error máximo de la interpolación lineal sobre un seno es $A(2\pi/N)^2/8 = 0{,}0006$ LSB, despreciable. Medido en el test con el código real: **0,99 LSB máximo a 10 Hz** (es el redondeo de la tabla más el de la salida), contra 2,96 LSB del camino directo.

La cuadrada y las sierras **no** se interpolan. Sus flancos se convertirían en rampas de $1/N$ del período (244 μs a 1 Hz). Por encima de 244 Hz se usa el camino directo, más rápido, porque el error de truncamiento ya queda al nivel del ruido del DAC. El umbral se cambia con `DDS_INTERP_MAX_STEP`.

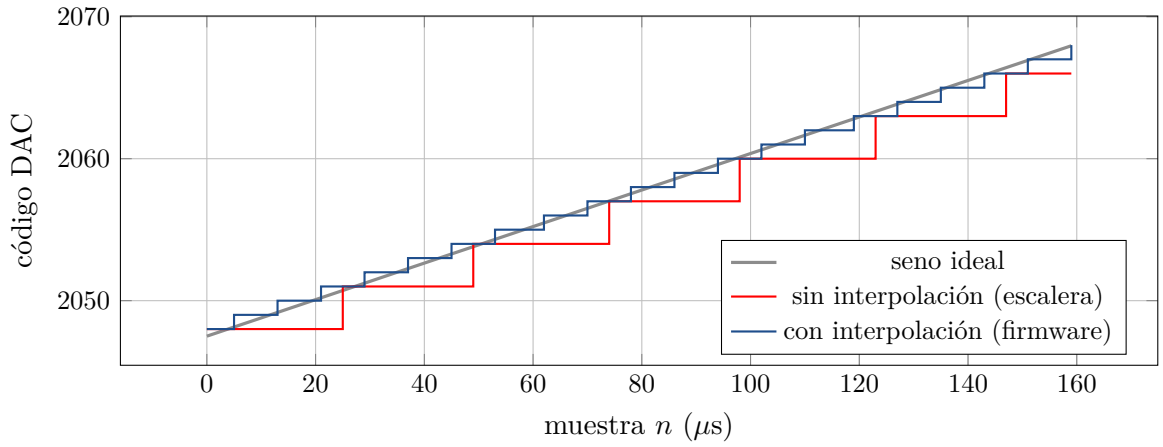


Figura 2: Seno de 10 Hz cerca del cruce por cero, salida real de `dds_fill()`. Sin interpolar, cada muestra de tabla se repite 24 veces (saltos de ≈ 3 LSB). Interpolando, la salida avanza de a 1 LSB.

3.5. Presupuesto de CPU y memoria

Camino (CH2 en 90°)	Instr./muestra	Ciclos (est.)	CPU 1 MS/s	CPU 5 MS/s
Directo	10	≈ 12	≈ 5 %	≈ 25 %
Interpolado	22	≈ 26	≈ 10 %	≈ 52 %

A 1 MS/s hay 250 ciclos por muestra. El firmware mide el peor caso real con el contador DWT y lo muestra por UART (`CPU x.x%`). RAM: 16 KB de tablas + 2 KB de buffer + pila $\approx$ 20 KB de 32 KB. Flash: 18,5 KB de 128 KB.

4. Diagramas de flujo

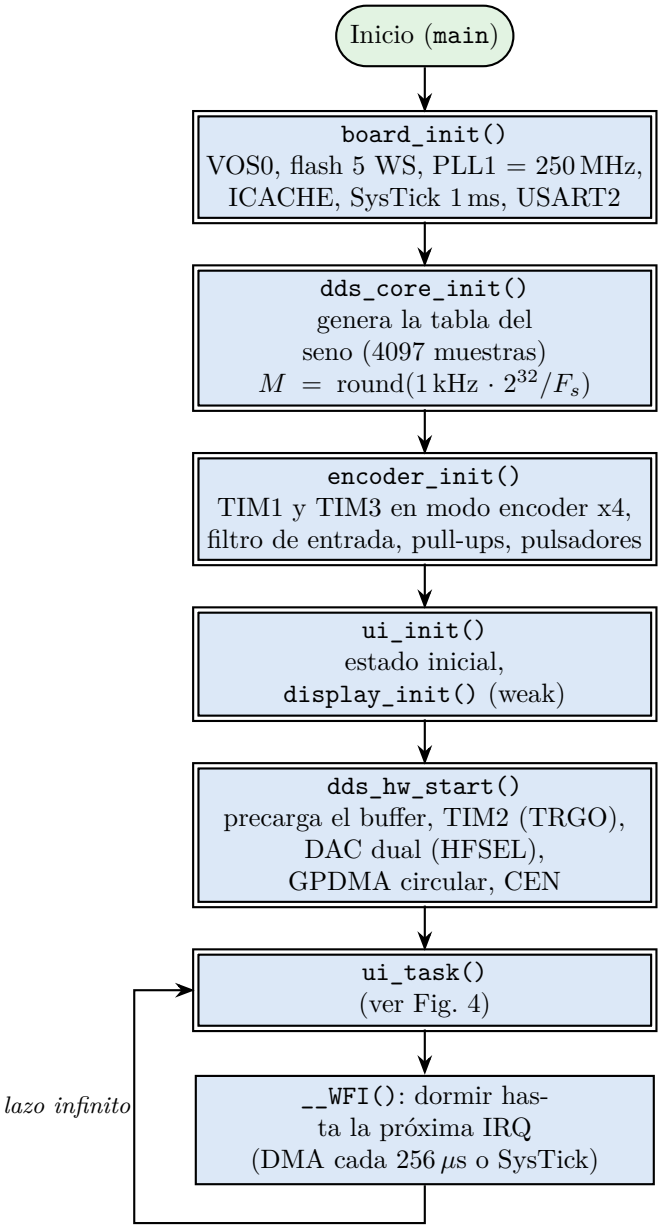


Figura 3: Programa principal.

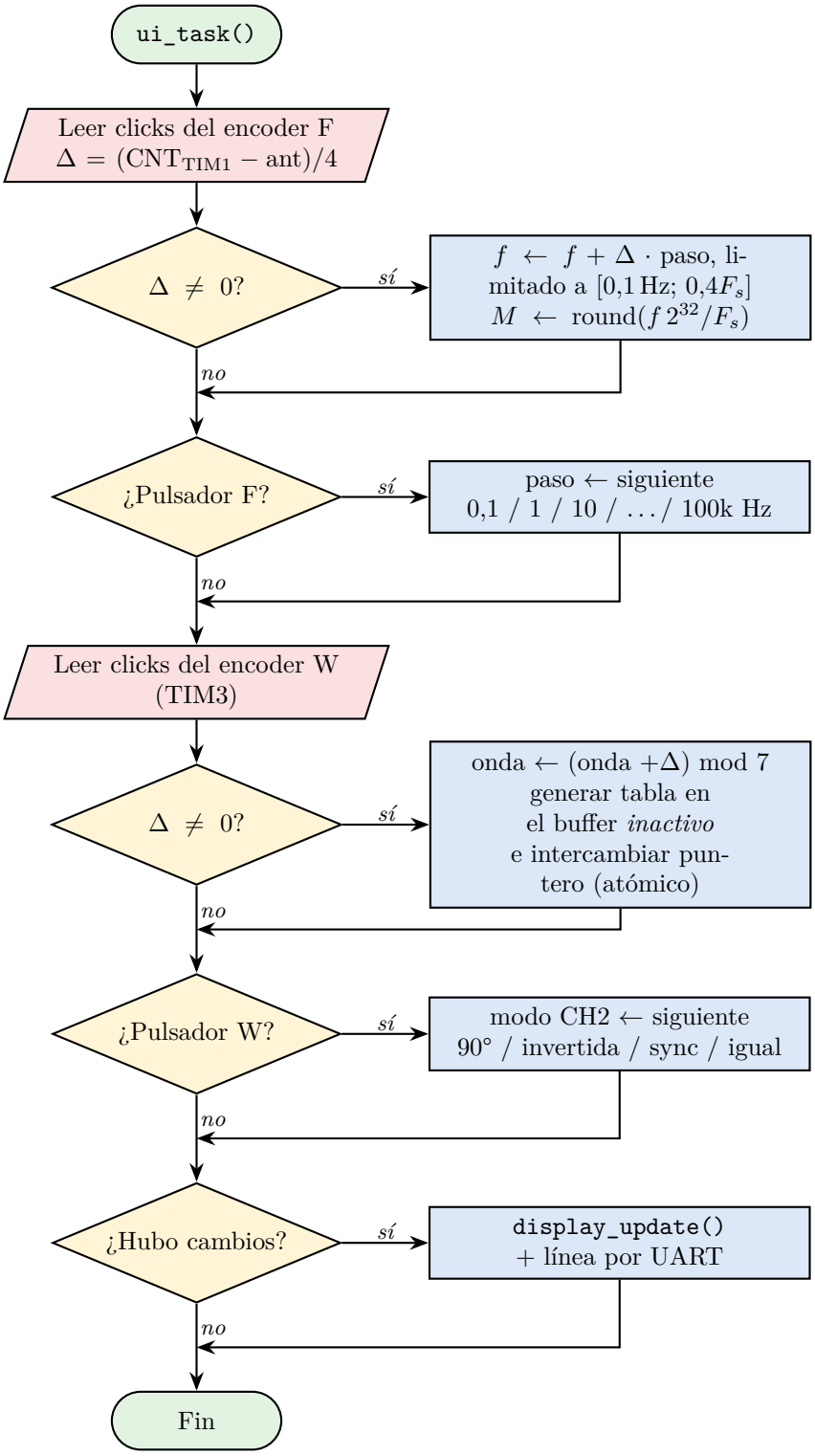


Figura 4: Lazo de interfaz: los encoders cuentan por hardware y `ui_task()` solo lee la diferencia. Los cambios de M entran en el borde del bloque siguiente, con fase continua.

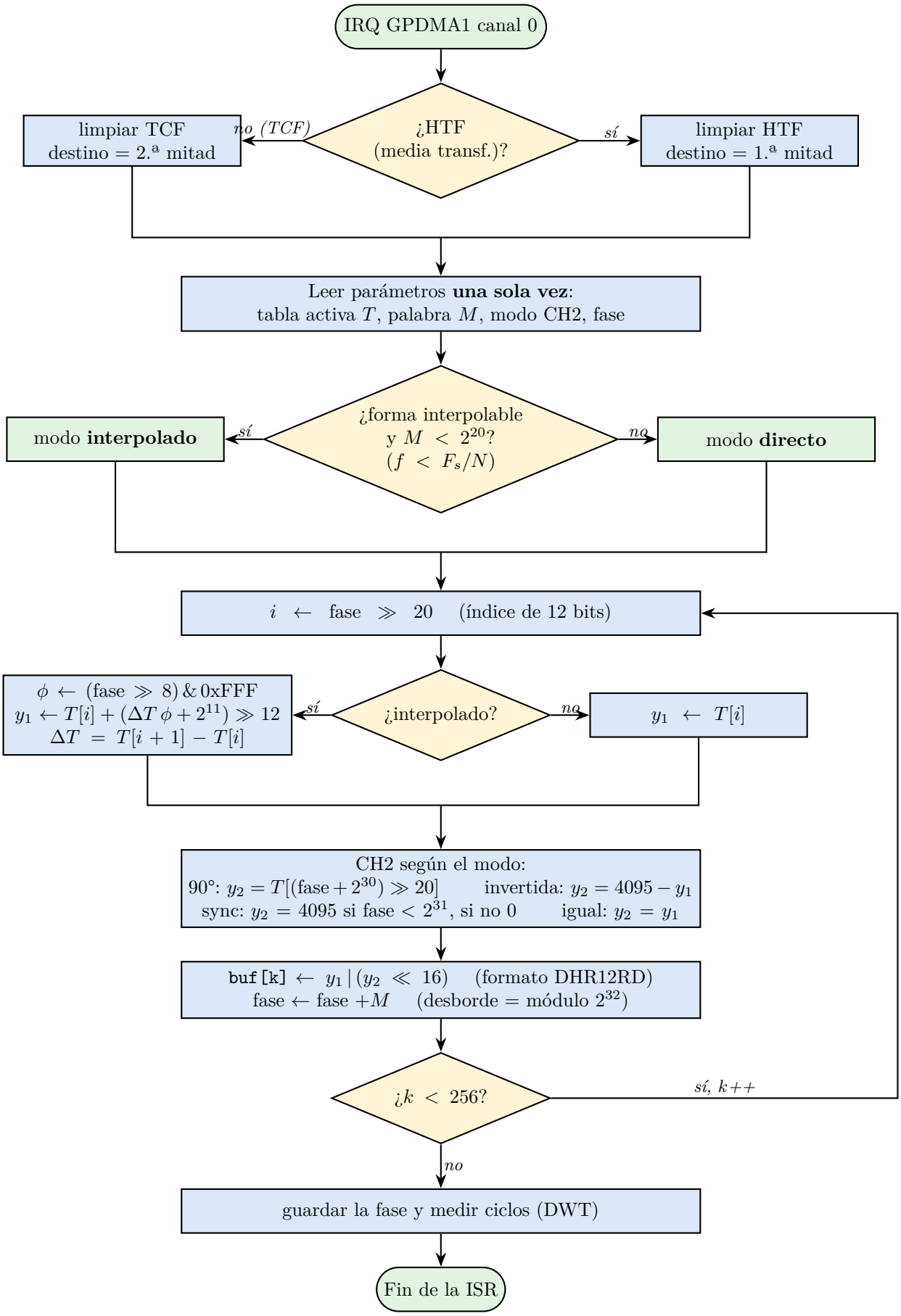


Figura 5: Motor DDS (ISR del GPDMA). En el código, la decisión "¿interpolado?" y el modo de CH2 se resuelven *antes* del lazo: el compilador genera 8 lazos especializados sin ningún `if` adentro.

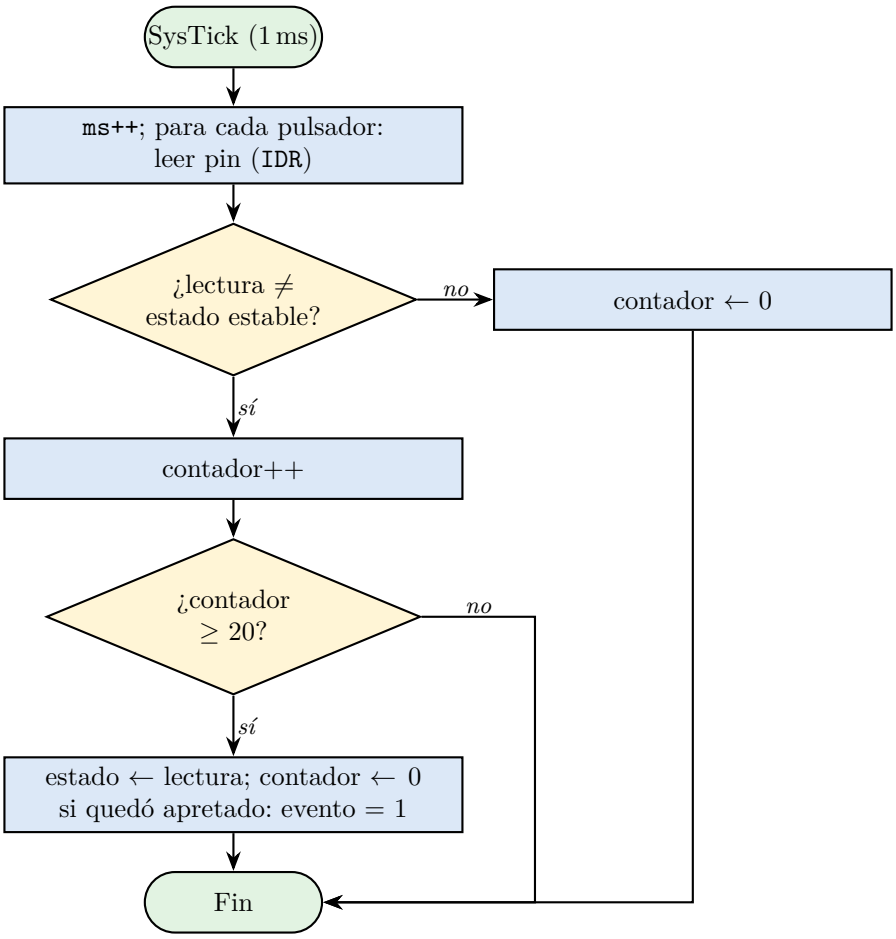


Figura 6: Antirrebote de los pulsadores por integración (20 ms estables).

5. Conexionado (LQFP48)

Pin	Función	AF	Conexión
PA4	DAC1_OUT1	analóg.	Salida CH1 → filtro → BNC
PA5	DAC1_OUT2	analóg.	Salida CH2 → filtro → BNC
PA8 / PA9	TIM1_CH1 / CH2	AF1	Encoder FRECUENCIA (A / B)
PA10	GPIO entrada, pull-up	–	Pulsador FRECUENCIA (cambia el paso)
PA6 / PA7	TIM3_CH1 / CH2	AF2	Encoder FORMA (A / B)
PB0	GPIO entrada, pull-up	–	Pulsador FORMA (modo de CH2)
PA2	USART2_TX	AF7	Depuración 115200 8N1 (opcional)
PA13 / PA14	SWDIO / SWCLK	–	ST-LINK
PB6 / PB7	I2C1 SCL / SDA	AF4	*Reservado:* pantalla I^2C (SSD1306)
PB13/15/12	SPI2 SCK/MOSI/CS	AF5	*Reservado:* pantalla SPI (ST7735, ILI9341)
PB14 / PB10	GPIO	–	*Reservado:* D/C y RESET de la pantalla

Recomendaciones de hardware. Encoders con el común a GND y una red RC de 10 kΩ/10 nF en A y B. En el LQFP48, V_{REF+} está unido a V_{DDA}: alimentá V_{DDA} con un filtro LC aparte para bajar el ruido del DAC. Con el buffer interno activado la salida no llega a los rieles (unos 0,2 V de cada lado). Para amplitud y offset ajustables agregá una etapa con un amplificador operacional después del filtro reconstructor.

Pantalla futura. `ui.c` define `display_init()` y `display_update(const ui_state_t*)` como `weak` (vacías). Basta con crear `display.c` con esas dos funciones: reciben la frecuencia pedida y la real, el paso (para dibujar el cursor), la forma de onda, el modo de CH2 y si la interpolación está activa. El motor DDS no se toca.